

BREAKOUT ARCADE GAME

Technical Documentation

SUBMITTED BY: Abdullah Saleem // 24F-0530 // CS-3E

TABLE OF CONTENTS

1. [Game Overview](#)
2. [System Flowchart](#)
3. [Interrupt Usage](#)
4. [Scoring and Gameplay Mechanics](#)
5. [Key Functions](#)

1. GAME OVERVIEW

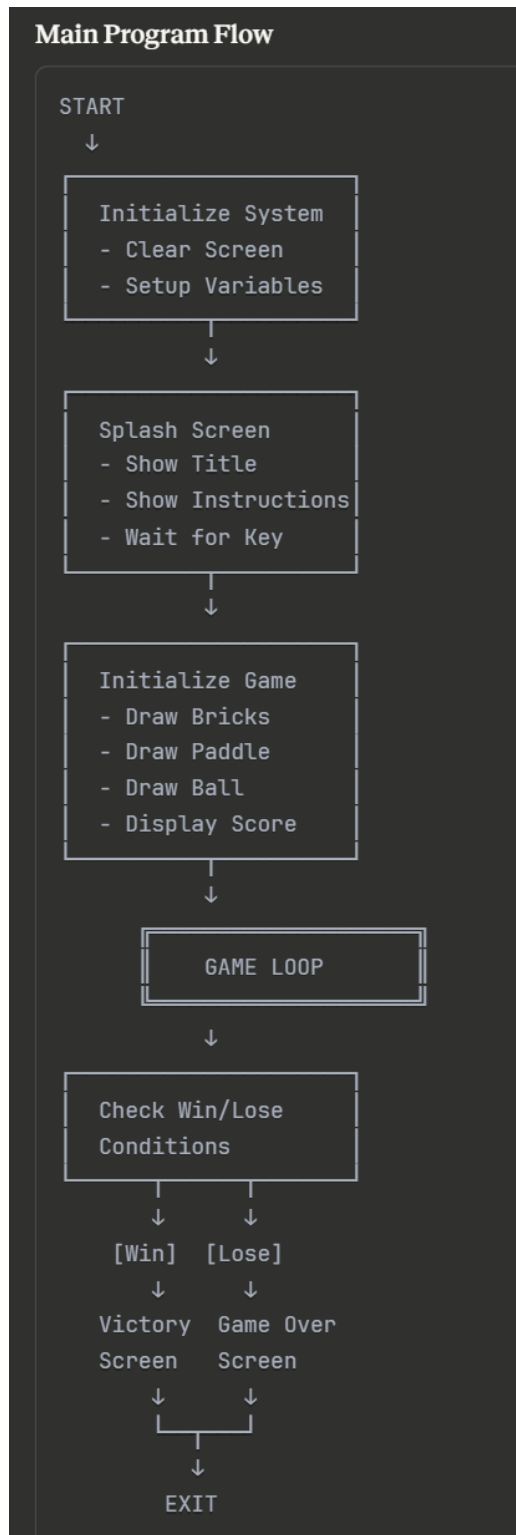
Game Description

A classic Breakout-style arcade game implemented in x86 Assembly Language. The player controls a paddle to bounce a ball and destroy colored brick tiles. The game features multiple difficulty levels, dynamic scoring, sound effects, and win/lose conditions.

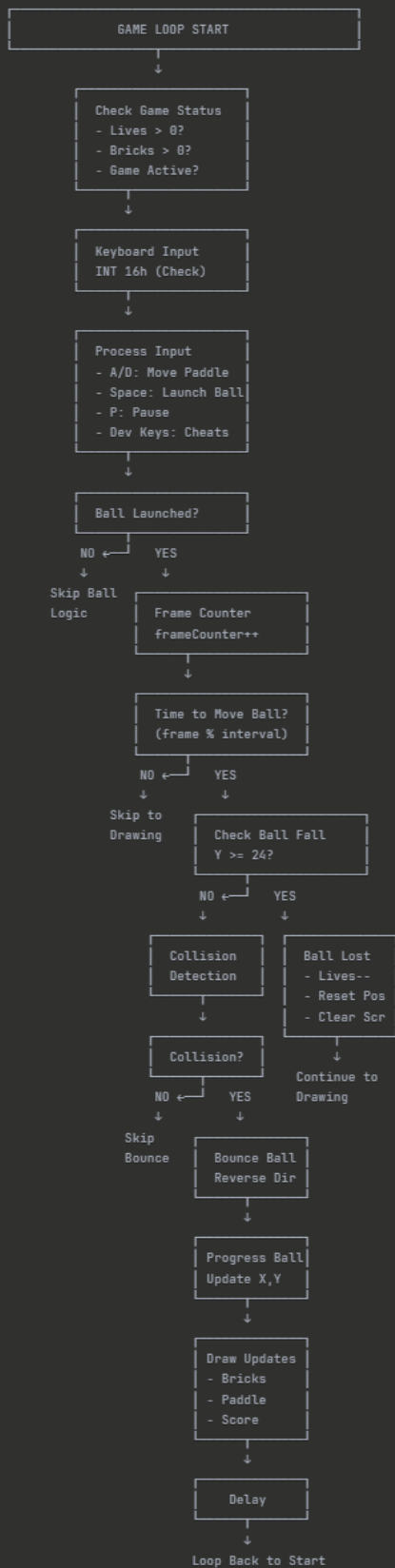
Core Features

- **Visual Elements:** Paddle, Ball, 40 Colored Bricks (4 rows × 10 columns)
- **Controls:** A/D keys for paddle movement, Spacebar to launch ball, P to pause
- **Lives System:** 3 lives; lose one when ball falls below paddle
- **Pause System:** P key to Pause and Resume the game
- **Win Condition:** Destroy all 40 bricks
- **Lose Condition:** Run out of lives
- **Sound Effects:** Ball bounce, paddle hit, brick destruction, game over/victory melodies
- **Developer Keys:** Speed control (3/4), paddle width (5/6), instant win/lose (1/2)

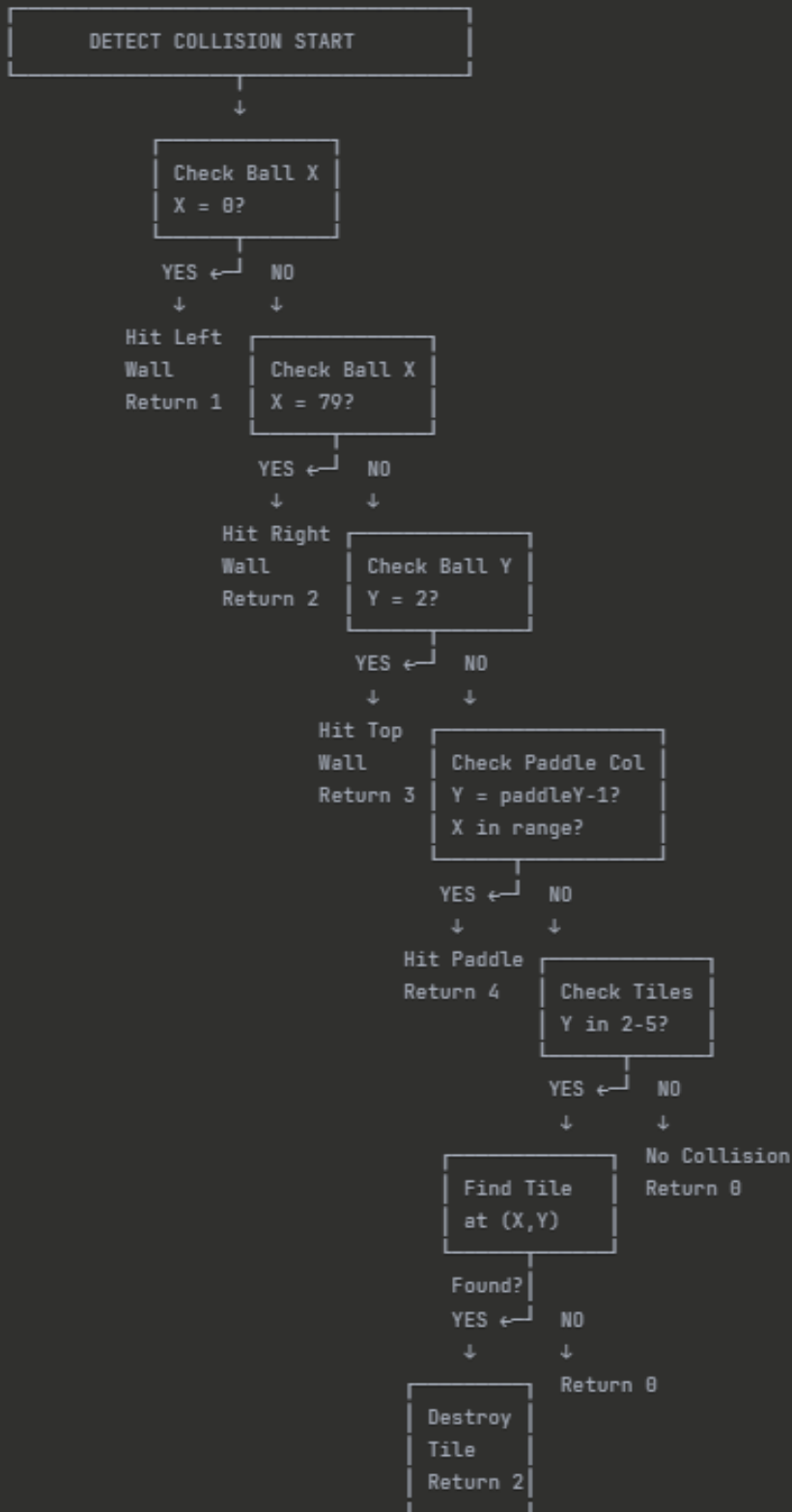
2. SYSTEM FLOWCHART



Game Loop Detailed Flow



Collision Detection Flow



3. INTERRUPT USAGE

INT 16h - Keyboard Services

Purpose: Keyboard input handling

Used Functions:

1. AH = 01h - Check Keyboard Status

- Checks if key is available without removing it from buffer
- Sets ZF if no key pressed

2. AH = 00h - Read Keyboard

- Waits for and reads keypress
- Returns: AL = ASCII character, AH = Scan code
- Clears keyboard buffer

INT 21h - DOS Services

Purpose: Program termination

Used Function:

- **AH = 4Ch - Terminate Program**
 - AL = Return code (0 = success)
 - Usage: `mov ax, 0x4C00; int 0x21`
 - Called at: Game over, victory, or user exit

PC Speaker Sound (Port I/O)

Purpose: Sound effects generation

Ports Used:

- **Port 0x43:** Timer control port
- **Port 0x42:** Timer counter
- **Port 0x61:** PC speaker control

Sound Effects:

- **Ball Bounce:** Short high-pitched beep
- **Paddle Hit:** Medium-pitched beep
- **Brick Destruction:** Descending tone
- **Victory:** Ascending melody
- **Lose:** Descending melody
- **Dev Key:** Quick chirp

4. SCORING AND GAMEPLAY MECHANICS

Scoring System

Dynamic Score System. Bricks of each color have different score.

Maximum Possible Score: 1000 points

- Row 0: 10 bricks $\times$ 40 = 400
- Row 1: 10 bricks $\times$ 30 = 300
- Row 2: 10 bricks $\times$ 20 = 200
- Row 3: 10 bricks $\times$ 10 = 100
- **Total:** 1000 points

Lives System

Starting Lives: 3

Losing a Life:

1. Ball Y coordinate $\geq$ 24 (falls below paddle)
2. Lives counter decremented
3. Ball and paddle positions reset
4. Ball launch status reset (must press spacebar again)
5. Screen cleared and redrawn

Game Over Condition: Lives = 0

Brick System

Total Bricks: 40 (4 rows × 10 columns)

Brick Data Structure (5 bytes each):

```
Offset 0: X position (byte)
Offset 1: Y position (byte)
Offset 2: Width (byte)
Offset 3: Status (byte) - 1=active, 0=destroyed
Offset 4: Attribute (byte) - color
```

Brick Layout:

- Row spacing: 1 unit vertically
- Variable widths (4-11 characters) for visual variety
- Staggered horizontal positions

Destruction Process:

1. Ball collision detected at (X, Y)
2. Calculate row: row = (Y - 2)
3. Find brick in row at X coordinate
4. Set brick status to 0
5. Decrement '**aliveBricks**' counter
6. Calculate and add score
7. Play destruction sound
8. Flag bricks for redraw

Ball Mechanics

Ball Data Structure:

ballData:

```
db X, Y          ; Position (bytes 0-1)
db deltaX, deltaY ; Movement vector (bytes 2-3)
db attribute      ; Color (byte 4)
```

Initial State:

- Position: (45, 22) - on paddle
- Movement: (1, -1) - right and up
- Status: Not launched

Movement Update (Frame-based):

```
if (frameCounter % ballMoveInterval == 0):
    newX = currentX + deltaX
    newY = currentY + deltaY
    ballPosition = (newX, newY)
```

Speed Control (Developer Keys 3/4):

- ballMoveInterval controls frames between moves
- Lower value = faster ball
- Higher value = slower ball
- Default: 5 frames

Paddle Mechanics

Paddle Data Structure:

paddleData:

```
db X, Y          ; Position (bytes 0-1)
db width         ; Width (byte 2)
db attribute      ; Color (byte 3)
```

Initial State:

- Position: (35, 23)
- Width: 10 characters
- Y position fixed (never changes)

Movement:

- 'A' key: X-- (move left)
- 'D' key: X++ (move right)
- Boundary checks: $0 \leq X \leq 80 - \text{width}$

Width Control (Developer Keys 5/6):

- Key 5: Decrease width (minimum: 4)
- Key 6: Increase width (maximum: 20)
- Affects collision detection range

6. KEY FUNCTIONS

Core Game Functions

clearscreen

- Fills entire screen with spaces (0x0720)
- Uses rep stosw for efficiency
- 2000 characters (80×25)

putChar(x, y, char+attr)

- Places single character at screen position
- Calculates offset: $(Y \times 80 + X) \times 2$
- Writes to video memory 0xB800

drawSingleBrick(brickAddr)

- Draws one brick at specified address
- Checks status (active/destroyed)
- If destroyed: draws spaces
- If active: draws brick character

drawAllBricksIndividually

- Iterates through 40 bricks
- Calls drawSingleBrick for each
- Resets needToPrintBricks flag

drawThePaddle

- Draws paddle at current position
- Width-aware (variable paddle size)
- Resets needToPrintPaddle flag

drawBall

- Draws ball at (ballX, ballY)
- Single character 'O'

removeBall / removeThePaddle

- Overwrites with spaces
- Called before position update

Input Functions

processKeyboardInput(keyASCII)

- Routes key to appropriate handler
- A/D: paddle movement
- Space: ball launch
- P: pause
- 0: exit
- 1-6: developer commands

Physics Functions

progressBall

- Removes ball at old position
- Updates X by deltaX
- Updates Y by deltaY
- Draws ball at new position

detectCollision

- Returns collision code (0-4)
- Checks walls, paddle, bricks
- Priority: walls → paddle → bricks

checkPaddleCollision

- Y check: ballY == paddleY-1
- X check: ballX in [paddleX, paddleX+width)
- Returns 4 if hit, 0 otherwise

checkTileCollision

- Bounds check: Y in [2, 5]
- Calls getTheTileOffset
- If tile found: destroyTheTile
- Returns 2 if hit, 0 otherwise

getTheTileOffset(X, Y)

- Calculates row: row = Y - 2
- Offset = row × 10 × 5
- Searches 10 tiles in row
- Returns SI = tile address or 0

destroyTheTile(tileAddr)

- Sets tile status to 0
- Decrements aliveBricks
- Calculates row-based score
- Adds to score total
- Plays destruction sound

bounceBall(collisionCode)

- Code 1 or 2: reverse deltaX
- Code 3 or 4: reverse deltaY

Game State Functions

checkBallFell

- Checks if ballY $\geq$ 24
- Returns 1 if fell, 0 otherwise

resetBallAndPaddle

- Paddle: (35, 23)
- Ball: (40, 22)
- deltaX = 1, deltaY = -1
- ballLaunchStatus = 0

printScoreAndLives

- Converts numbers to ASCII
- Displays at top of screen
- "SCORE: XXX" at (0, 2)
- "LIVES: X" at (0, 65)

showGameOver

- Clears screen
- Displays "GAME OVER!" in red

- Shows final score
- Plays lose melody
- Waits for keypress

showVictory

- Clears screen
- Displays "YOU WIN!" in green
- Shows final score
- Plays victory melody
- Waits for keypress

CONCLUSION

This Breakout game demonstrates fundamental assembly language concepts including:

- Direct hardware manipulation (video memory, ports)
- Interrupt handling (keyboard, video, DOS)
- Game loop architecture
- Collision detection algorithms
- State management
- Sound generation
- Frame-based timing

The modular function design allows for easy maintenance and expansion while maintaining efficient execution in the constrained x86 real mode environment.
